

3 Het 'ECOL' - rapport

In het nu volgende hoofdstuk vindt U een volledige beschrijving van de programmeertaal ECOL. Iets te volledig zelfs, want in enkele opzichten is men vooruit gelopen op de situatie. Met name het FUN-mechanisme en de MET-opdracht zijn op het moment van het verschijnen van dit boek nog niet in werkelijkheid toe te passen.

Dit hoofdstuk is ook onafhankelijk van dit docentenboek, als rapport, verschenen. Daarom wijkt de paragraafnummering af van die van overige hoofdstukken. Zelfs is dit hoofdstuk onderverdeeld in zes hoofdstukken, aangegeven met Romeinse cijfers. Hieronder vindt U een inhoudsopgave. Aan het eind van dit hoofdstuk vindt U ook een index om het zoeken te vergemakkelijken.

- INHOUD -

I	Taaldefinitie, vertaling, executie	15	Lege opdracht
1	Taaldefinitie	16	Invoer
2	Vertaling	16.1	LEES, LEESYM
3	Executie	16.2	BESTAND (x,x,x)
II	Globale vorm van ECOL programma's	16.3	BESTAND (x)
4	Structuur van een ECOL programma	17	Uitvoer
5	Labels, kommentaar, spaties	17.1	SCHRIJF (x,x)
5.1	Labels	17.2	SCHRIJF
5.2	Kommentaar	17.3	SCHRIJM
5.3	Spaties	17.4	SCHRIJFSYM
III	Context-regels en compiler actie	17.5	TEKST
6	Geheugenplaatsen, waarde, adres, descriptor	17.6	SPATIE(x)
7	Namen	17.7	SPATIE
8	Labels en sprongopdrachten	17.8	NR(x)
IV	Executie	17.9	NR
9	Inleidende opmerkingen	17.10	NP
10	Uitdrukkingen	17.11	BESTAND (x,x,x)
10.1	Getal	17.12	BESTAND (x)
10.2	Variabele	17.13	Opmerkingen uitvoer
10.3	Fun-roep	18	Standaardfuncties
10.4	Waarden van uitdrukkingen; prioriteiten	V	SUB en FUN
10.5	Volgorde bepaling primitiva	19	Inleiding
10.6	Waarde van aMb	20	SUB-beschrijving
11	Toekenning	21	SUB-roep
12	Sprongopdrachten	22	Informeel inleiding FUN mechanisme
13	RIJ-reservering	23	FUN-beschrijving
14	Cyclus	24	FUN-roep
		25	Recursiviteit
		VI	De ECOL syntax

DEFINIEREND RAPPORT VAN ECOL, EEN PROGRAMMEERTAAL VOOR HET VOORTGEZET ONDERWIJS

ECOL, samentrekking van Educational COmputer Language, is de programmeertaal die o.a. wordt gebruikt in de leergang "Computerkunde 1 en 2 voor a.v.o. en v.w.o." door C.A.Ch.Görts, S.G.van der Meulen, A.van der Sluis en J.R.Zweerus, uitgave Wolters-Noordhoff, Groningen 1970-1971.

Aan de opstelling van dit rapport werd, behalve door de vier auteurs van de genoemde leergang, meegewerkt door E.Lopez Cardozo, J.H.J.M.Truijens, G.A.Vonk.

I TAALEDEFINITIE, VERTALING, EXECUTIE

1. Taaldefinitie

ECOL is een taal om programma's voor een computer te formuleren. Bij het beschrijven van zo'n taal moet men enerzijds zeggen hoe correcte teksten in deze taal er uit zien, anderzijds wat voor computer-akties deze teksten beschrijven.

Hoe een correcte ECOL-tekst er uit ziet wordt voor een belangrijk deel vastgelegd in een syntax (zie VI). Omdat deze, ter wille van de eenvoud, weinig formeel is gehouden, is hij in feite wat te ruim, en zijn verdere inperkingen nodig. Deze worden in dit rapport gegeven in zgn. context-regels. Verwijzingen naar syntax-regels worden gegeven door [], verwijzingen naar context-regels door C...(C 8.1 bijv. staat in paragraaf 8).

In het rapport wordt voortdurend naar de syntax verwezen, en de lezer wordt dan ook uitgenodigd te beginnen met de inleiding tot de syntax te lezen.

2. Vertaling

Bij de computerverwerking van ECOL-teksten zijn twee fasen te onderscheiden, nl. de vertaalfase en de executiefase.

In de vertaalfase wordt de ECOL tekst door een voor ECOL gemaakt vertaalprogramma ("compiler") regel na regel, van boven naar beneden, en per regel van links naar rechts, gelezen en vertaald naar interne machine-opdrachten. We duiden deze leesvolgorde verder aan als tekstuele volgorde. Tijdens deze actie ziet de compiler voortdurend uit naar overtredingen van syntax en context-regels. Wanneer hij zo'n fout ontdekt wordt deze signaleerd, zoveel mogelijk met regelnummer en aanduiding wat er fout is. Na het ontdekken van een fout gaat de compiler rustig verder met lezen om te trachten nog meer fouten te ontdekken, zodat de auteur van het programma in één keer zoveel mogelijk fouten aan de weet komt. Evenwel kan een fout tot gevolg hebben dat foutmeldingen gegeven worden betreffende andere regels ECOL-tekst die in feite goed zijn.

Het is echter duidelijk dat als er eenmaal een fout is geconstateerd, niet tot executie van het programma kan worden overgegaan. Ook in de beschrijving van ECOL op de volgende pagina's duikt de compiler af en toe op. Strikt genomen is dit overbodig. Het kan echter bijdragen tot goed begrip van wat er gebeurt.

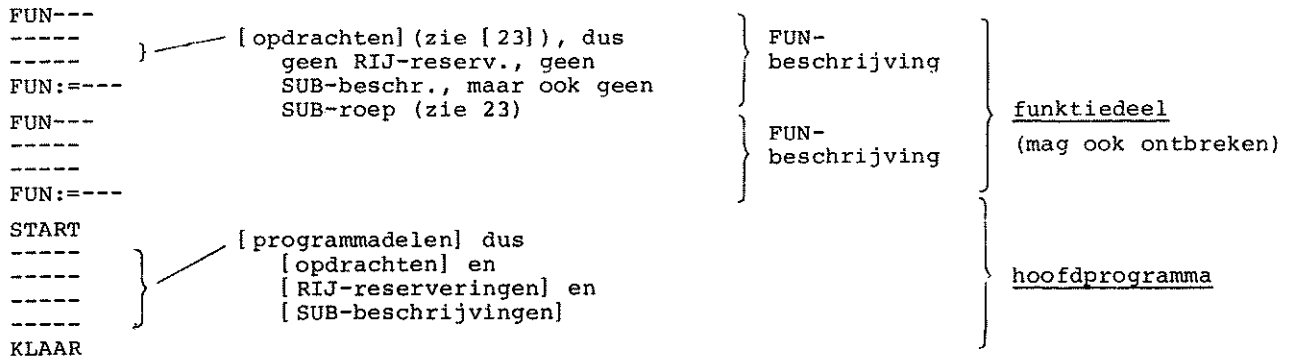
3. Executie

Als het programma bij vertaling "correct" bevonden is, wordt tot uitvoering (executie) overgegaan, d.w.z.: de computer gaat nu doen, wat de programmeur in zijn (taalkundig correct) programma voorschreef. Dit hoeft evenwel niet te betekenen, dat er nu ook gebeurt wat de programmeur beoogde. Er kan bijvoorbeeld iets uit een GP worden gehaald, voordat er iets in is gestopt; er kunnen RIJ-grenzen worden overschreden; er kan gevraagd worden een getal te lezen terwijl er geen getallen meer zijn, enz. Allemaal fouten die een vertaal-programma onmogelijk kan voorzien. We noemen dit executie-fouten; ook hiervan worden er vele signaleerd, weer met vermelding van regelnummer. Meestal heeft een executie-fout tot gevolg dat het programma wordt afgevoerd, d.w.z.: niet verder wordt geëxecuteerd.

II GLOBALE VORM VAN ECOL-PROGRAMMA'S

4. Structuur van een ECOL-programma

Uit de syntax (zie [20]) blijkt dat een ECOL-programma er als volgt uitziet.



Het funktiedeel, waarover in de leergang Computerkunde niet gesproken wordt, maakt het mogelijk, onafhankelijk van het hoofdprogramma een of meer rekenschema's (bestaande uit gewone ECOL-opdrachten "klaar te zetten", die dan door het hoofdprogramma kunnen worden gebruikt. Voor nadere verduidelijking zie 22 en volgende.

De indeling van een programmatekst in regels wordt in hoge mate door de syntax voorgeschreven. Overgang op een nieuwe regel programmatekst is in ECOL dan ook zeer betekenis-dragend. Zo mag men niet twee korte opdrachten op één regel zetten, of een lange formule op een volgende regel voortzetten.

5. Labels, kommentaar, spaties

5.1. Labels

Uit de syntax blijkt (zij het op een nogal ingewikkelde wijze), dat elke regel programma-tekst mag beginnen met hoogstens één [label], dat is een geheel getal zonder teken (zie [14]), eventueel gevolgd door dubbelpunt (zie [15]).

5.2. Kommentaar

Uit de syntax blijkt dat achteraan elke regel tekst een puntkomma, eventueel gevolgd door willekeurige symbolen mogen komen. Wat er achter de puntkomma staat is het zgn. kommentaar. De enige functie ervan is dat dit bij het afdrucken van de programmatekst mee wordt afgedrukt. Verder wordt er niets mee gedaan. Het dient om de tekst voor de menselijke lezer wat leesbaarder te maken.

In een programmatekst mogen op willekeurige plaatsen regels voorkomen die hetzij helemaal leeg zijn of alleen een label en/of kommentaar bevatten. Dit mag omdat [86] een "lege" FUN-beschrijving toelaat en [23] een "lege" opdracht. Wegens C 8.1 zijn daarom de labels op dergelijke regels tussen twee "echte" FUN-beschrijvingen geheel willekeurig, terwijl op dergelijke regels binnen een FUN-beschrijving of binnen het hoofdprogramma de labels moeten verschillen van de overige labels in de betreffende FUN-beschrijving resp. het hoofdprogramma.

5.3. Spaties

In ECOL-teksten mogen bijna overal spaties worden ingelast of weggelaten, zonder dat daardoor de betekenis, dat wil zeggen: de bij executie geproduceerde uitvoer, of de afgegeven foutmeldingen, verandert. De uitzonderingen staan in de inleiding tot de syntax:

- a) g e e n spaties mogen worden ingelast in hoofdletter-woorden (zoals START, LEES, ALS etc.);
- b) g e e n spaties mogen worden toegevoegd of weggelaten in een rijtje symbolen tussen aanhalings-teken, zoals achter "TEKST :=" kan voorkomen (dat is niet fout, maar daardoor verandert de geproduceerde uitvoer en dus de betekenis van het programma);
- c) tenminste 1 spatie m o e t er zijn wanneer hoofdletter-woorden op elkaar volgen.

Voorbeeld 5.1

```

7                ; demonstratietekst voor labels, kommentaar en spaties.
8  START
7:              ; deze nieuwe label 7 is toegestaan (zie 5.2 en C 8.1).
15 2            ; dit is label 152.
5 : x:  = LEES   ; lelijk "wordt-teken" maar wel goed.
    y := LEE S   ; fout, spaties in hoofdletterwoord.
9  ALS SIN(x) > 0.5 DAN 5 ; goed; spatie tussen ALS en SIN. In plaats van SIN(x) had er ook
                        ; mogen staan SIN ( x ).
5  TEKST := "jan hein" ; als men de spatie tussen jan en hein weglaat wordt er afgedrukt
                        ; janhein.
8  KLAAR        ; fout, label 8 was al uitgegeven in het hoofdprogramma; verder
                        ; mag op de regels volgend op KLAAR geen kommentaar meer komen.

```

III CONTEXT-REGELS EN COMPILER-AKTIE

6. Geheugen-plaatsen, waarde, adres en descriptor

Zoals in "Computerkunde I" staat, is een computer een apparaat dat getallen snel kan optellen, aftrekken, vermenigvuldigen, delen en vergelijken. We weten dat die getallen in het geheugen zijn opgeslagen; hoe dat technisch is gerealiseerd, doet hier niet ter zake. Wel van belang voor ons is:

Een plaats in het geheugen waar een getal kan worden opgeborgen, noemen we een Geheugen-Plaats (afgekort: GP).

Alle GP's in het geheugen zijn genummerd, het nummer van een GP heet zijn adres.

Het onderscheid tussen het getal in de GP (ook wel de inhoud of de waarde van de GP genaamd) en het adres van de GP speelt op verschillende plaatsen een rol in deze beschrijving van ECOL.

Aan elke [variabele] van de vorm [GP] (zie syntax [32]) in het hoofdprogramma wordt door de compiler een GP (dus een adres) toegekend.

Bij RIJ-en ligt de zaak ingewikkelder. Als men schrijft

RIJ(m:n)r of RIJSYM(m:n)r

dan krijgt men bij executie n-m+1 GP's ter beschikking:

r(m), r(m+1), -----r(n-1), r(n)

en elk van deze GP's heeft natuurlijk een adres. Maar de compiler weet niet welke adressen dit zijn omdat hij niet weet welke waarden m en n tijdens executie zullen hebben. Door sprongopdrachten kan het voorkomen dat een RIJ-reservering meerdere keren geëxecuteerd wordt met telkens andere onder- en bovengrens. Daarom zet de compiler voor elke rij een descriptor klaar, waarin tijdens executie geadministreerd kunnen worden

a) het begin-adres van de RIJ (dat is het adres van het eerste element)

b) de waarde van de onder-grens en de boven-grens,

c) gegevens die verder van belang kunnen zijn (zoals het al of niet gereserveerd zijn van de RIJ)

en hij vertaalt de RIJ-reservering in een aantal opdrachten die bij executie de nodige gegevens in de descriptor noteren. Een RIJ-reservering is dus wel degelijk een aktie.

Uit de syntax (zie [55]) blijkt nog dat onder- en bovengrens van een RIJ-reservering willekeurige [uitdr]'s (= uitdrukkingen, zie [34]) mogen zijn, zodat dus bijv. RIJ (p+1 : rxs)q toegestaan is.

7. Namen

Uit de syntax-regels [32], [33], [72] en [82] blijkt dat in een ECOL-programma verschillende soorten namen kunnen voorkomen:

[GP], [GP-rij], [SUB-naam] en [FUN-naam]

De schrijver van het programma weet natuurlijk wat er op elke plaats met een zekere [naam] wordt bedoeld, maar de lezer (in casu de compiler) weet dit niet. Vandaar de volgende context-regels

waaruit blijkt dat bij het in tekstuele volgorde lezen van een ECOL-tekst van elke [naam] die men ontmoet, direkt bekend is wat voor soort [naam] het is, ook als die [naam] nog niet eerder is voorgekomen. We volstaan hieronder met de context-regels voor het hoofdprogramma; de overeenkomstige regels voor het functie-deel vindt men in 23.

Context-regels voor het hoofdprogramma

C7.1 Een en dezelfde [naam] heeft overal in het hoofdprogramma dezelfde betekenis.

Gevolgen

- a) Een [naam] mag niet gebruikt worden in meer dan één der betekenissen [GP], [GP-rij], [SUB-naam], [FUN-naam].
- b) Als een [naam] een
[GP] is stelt hij overal dezelfde GP voor
[GP-rij] is stelt hij overal dezelfde RIJ voor
[SUB-naam] is verwijst hij overal naar dezelfde [SUB-beschrijving]
[FUN-naam] is verwijst hij overal naar dezelfde [FUN-beschrijving].
- c) Een [naam] mag niet meer dan eens op de plaats van [GP-rij], [SUB-naam] of [FUN-naam] in een [RIJ-reservering], [SUB-beschrijving] of [FUN-beschrijving] optreden.

C7.2 Een [naam] is een [FUN-naam] als hij eerder in de programmatekst op de plaats van [FUN-naam] in een [FUN-beschrijving] is opgetreden (zie [86]).

C7.3 Een [naam] is een [GP-rij] als hij eerder in de programmatekst op de plaats van [GP-rij] in een [RIJ-reservering] is opgetreden (zie [55]).

Gevolg

In de programmatekst mag een element van een zekere RIJ pas voorkomen nadat de reservering van deze RIJ verschenen is.

C7.4 Een [naam] is een [SUB-naam] als hij optreedt als [SUB-roep] of op de plaats van [SUB-naam] in een [SUB-beschrijving] (zie [70] en [71]). Een [SUB-naam] moet ook eenmaal op de plaats van [SUB-naam] in een [SUB-beschrijving] optreden.

Gevolg

In de programmatekst mag een [SUB-roep] voorkomen voordat de betreffende [SUB-beschrijving] is verschenen. N.B. Een [SUB-roep] is in de programmatekst gemakkelijk te herkennen: het is een [naam] die als zelfstandige [opdracht] optreedt (zie [70] en voorbeeld 7, regels 3 en 7).

C7.5 Een [naam] is een [GP] in alle overige gevallen.

Deze regels stellen bij een correct programma de compiler in staat alle optredende [namen] thuis te brengen. Wanneer tegen deze regels gezondigd wordt volgt een foutmelding, maar het kan nu verder voor de compiler wel moeilijk worden. Als een [naam] bijv. eenmaal verschenen is als [GP-rij] in een [RIJ-reservering] en vervolgens ook nog eens verschijnt zonder gevolgd te worden door een openingshaakje (dus in feite als [GP] of als [SUB-roep]), dan weet de compiler niet meer wat er nu eigenlijk bedoeld wordt (dit is een der oorzaken van het reeds in 2 gememoreerde verschijnsel dat een fout tot gevolg kan hebben dat "goede" stukjes tekst fout gemeld worden).

Voorbeeld 7.1

```

FUN f(x)    ; hiermee is "f" een [FUN-naam]
-----
-----
FUN :=w

START      ; hier begint het hoofdprogramma
a := LEES  ; hiermee is "a" een [GP]
RIJ(1:9)a  ; FOUT! "a" is al in gebruik genomen als [GP]
RIJ(1:a)r  ; in orde: "r" is hiermee een [GP-rij]
i :=a+3    ; hiermee is "i" een [GP]
y :=r(i)   ; hiermee is "y" een [GP], "r" wordt correct gebruikt als [GP-rij]
y :=u(i)   ; FOUT! "u" is nog niet verschenen in een [RIJ-reservering] en ook niet als
              ; [FUN-naam]
SUB klus   ; hiermee is "klus" een [SUB-naam]
-----
-----
END
y := klus  ; FOUT! "klus" wordt hier gebruikt als [GP], maar is al verschenen als
              ; [SUB-naam]
3 klus     ; in orde, een reeds beschreven SUB wordt geroepen
7 klos     ; GOED! "klos" verschijnt als een [SUB-roep], merk op dat het onmogelijk een
              ; [GP] kan zijn, "klos" is nu een [SUB-naam] en de compiler verwacht v ó ó r
              ; KLAAR de beschrijving van "klos"
y :=klos   ; dat is dus FOUT
SUB klos   ; daar is hij dan
-----
END
y :=f      ; FOUT! "f" is al verschenen als [FUN-naam] van een functie met een parameter
y :=f(a)   ; in orde, "f" wordt gebruikt in een [FUN-roep]
RIJ(1:a)f  ; FOUT! "f" is al verschenen als [FUN-naam] en staat niet meer ter beschikking
              ; als [GP-rij]
f          ; FOUT! "f" is al verschenen als [FUN-naam] van een functie met een parameter
KLAAR     ; dit programma komt nooit aan executie toe

```

8. Labels en sprongopdrachten

Zoals gezegd mag men elke programma-regel een label geven. Men is echter niet geheel vrij in de keuze van deze labels. Er geldt:

C8.1 Meerdere regels mogen hetzelfde getal als [label] hebben, mits maar alle labels in het hoofdprogramma verschillend zijn, evenals alle labels van een FUN-beschrijving (de labels in een SUB-beschrijving moeten dus ook verschillen van de labels in andere SUB-beschrijvingen en die in de rest van het hoofdprogramma).

Op deze regel is al geanticipeerd in 5.2 en voorbeeld 5.1.

Sprongopdrachten moeten verwijzen naar labels vooraan programmaregels. Ook hier is men weer niet geheel vrij. Er geldt:

C8.2 Een sprongopdracht die in het hoofdprogramma staat maar niet in een [SUB-beschrijving] mag alleen verwijzen naar een [label] in het hoofdprogramma maar niet in een [SUB-beschrijving].

C8.3 Een sprongopdracht die staat in een SUB- of FUN-beschrijving mag alleen verwijzen naar een label in dezelfde SUB- of FUN-beschrijving.

C8.4 Een sprongopdracht mag nooit verwijzen naar een [label] voor START, of voor een SUB- of FUN-beschrijving .

Als gevolg hiervan kan men een SUB of FUN alleen binnenkomen door de voordeur met een SUB- of FUN-roep, en alleen verlaten door de achterdeur met END resp. FUN:----

IV EXECUTIE

9. Inleidende opmerkingen

Wanneer de compiler een aangeboden ECOL-tekst correct heeft bevonden en heeft vertaald, kan het vertaalde programma in executie worden genomen, dat wil zeggen: worden uitgevoerd.

Bij executie van een ECOL-programma wordt begonnen met de [aktie] die tekstueel volgt op START. Na elke [aktie] wordt de tekstueel volgende [aktie] uitgevoerd, behalve bij sprongopdrachten, SUB- en FUN-roepen. Aangezien [SUB-beschrijvingen] geen [akties] zijn vallen zij buiten de executie-volgorde. Men mag dus rustig tussen twee [akties] een [SUB-beschrijving] zetten: die wordt bij executie overgeslagen. Men ziet dat de syntactische term [aktie] juist gekozen is: een [aktie] doet inderdaad iets, terwijl een [SUB-beschrijving] een rekenschema is dat door een [SUB-roep] geactiveerd moet worden.

In dit hoofdstuk wordt in meer detail besproken hoe een correct ECOL-programma wordt geëxecuteerd. We beginnen daarbij met te bespreken hoe waarden worden berekend uit formules. Daarna komen de verschillende akties ter sprake.

Afronden

In het volgende wordt nogal eens gesproken over afronding. Wanneer voor een willekeurig getal a en een geheel getal g geldt $g - \frac{1}{2} \leq a < g + \frac{1}{2}$ dan verstaan we onder het afronden van a het vervangen van a door g .

10. Uitdrukkingen

In [34] t/m [38] wordt recursief (voor de betekenis van dit woord zie inleiding syntax) vastgelegd wat een [uitdr] allemaal kan zijn. Zinnige samenstellingen van getallen, variabelen en FUN-roepen (de zgn. primitiva) met de operatoren $+$ $-$ $\times$ $/$ en M (machtsverheffing) en haakjesparen zijn toegestaan, mits geen twee van deze operatoren onmiddellijk op elkaar volgen. Dus niet $a \times -b$, ook niet $a \times -6$, maar wel $a \times (-b)$. Enige uitzondering: men mag i.p.v. M ook $\times \times$ schrijven.

Ingewikkeld voorbeeld:

$$r(i+j \times k) := ((a+b)/(a-b) - u \times v^2) - \sin(u+v) \times \cos(u-v) + r(i-j \times k)$$

Hier volgt eerst informatie over de primitiva en hun waarden.

10.1 Getal

Volgens [38] mag een getal genoteerd worden als geheel getal of als geheel getal gevolgd door punt (deze punt heeft dan geen betekenis) of als geheel getal gevolgd door punt gevolgd door decimale fractie.

Voorbeelden: 23 123. 3.47 Niet toegestaan is .47

10.2 Variabele

Er zijn twee soorten variabelen: [GP] of [GP-rij]([uitdr]), de laatste noemen we ook wel rij-element en de [uitdr] achter de [GP-rij] de index-expressie.

De waarde van een variabele is altijd de waarde die er bij executie het laatst aan is toegekend en die waarde is altijd een getal. Zoals bekend heeft een variabele behalve een waarde ook een adres.

Een GP-rij heeft evenveel adressen (en dus waarden) als is gegeven door ondergrens en bovengrens (zie 6). Het adres van een rij-element volgt uit de index, dat is de afgeronde waarde van de index-expressie. Er wordt gecontroleerd of deze index niet onder de ondergrens, of boven de bovengrens ligt; in dat geval volgt een foutmelding en wordt het programma afgevoerd. De waarde van een rij-element is het getal op het aldus aangewezen adres.

Voorbeeld

```
a := 2.3
b := 2.6
c := WRTL(2)      ; WRTL betekent vierkantswortel
r(a) := r(b) + r(c) ; dat is nu hetzelfde als : r(2) := r(3) + r(1)
```

Aan de schrijvers van compilers wordt aanbevolen ervoor te zorgen dat bij executie een foutmelding gegeven wordt zodra de waarde van een variabele gebruikt wordt voordat deze variabele in de loop van dit programma een waarde heeft gekregen.

10.3 FUN-roep

Bij een FUN-roep (zie ook hoofdstuk V) wordt het programma van de FUN-beschrijving doorlopen met de meegegeven aktuele(n). Een FUN-beschrijving eindigt altijd met een opdracht "FUN :=[uitdr] ". De waarde van [uitdr] is tevens de waarde die de FUN -roep aflevert.

10.4 Waarden van uitdrukkingen; prioriteiten

Wanneer de waarden der primitiva bekend zijn volgt de waarde van de uitdrukking door de operatoren + - x / M in de goede volgorde uit te voeren. Wat die goede volgorde is volgt uit onderstaande prioriteitsregels (die behalve ten aanzien van de deling overeenstemmen met Meneer Van Dalen Wacht Op Antwoord).

Omdat uitdrukkingen in een [vergelijking] (zie [50]) ook nog voorkomen in combinatie met vergelijkers, EN en OF geven we hier meteen een overzicht van al deze tekens in volgorde van hoogste naar laagste prioriteit; wat op dezelfde regel staat heeft gelijke prioriteit.

```
M
x /
+ -
< <= = > >
EN
OF
```

Gelijke prioriteit betekent dat de operatoren in tekstuele volgorde aan de beurt komen.

Voorbeelden van gelijke prioriteiten

```
a - b + c      betekent (a - b) + c
a / b x c      betekent (a / b) x c   N.B.  Wijkt af van Nederlands gebruik.
a M b M c      betekent (a M b) M c
```

Hogere prioriteit betekent dat een operator die tekstueel later komt dan een van lagere prioriteit, toch eerder wordt uitgevoerd.

Voorbeeld van verschillende prioriteiten

```
ALS a+bxc < pMq OF x = y-z EN u >= v/w DAN 12 ANDERS 13
betekent
```

```
ALS ((a+(bxc)) < (pMq)) OF ((x = (y-z)) EN (u >= (v/w))) DAN 12 ANDERS 13
```

Haakjesparen doorbreken uiteraard de prioriteitsregels: wat tussen haakjes staat moet eerst worden gedaan. Zodoende betekent $a + b \times c$ iets anders dan $(a + b) \times c$.

10.5 Volgorde bepaling primitiva

Met de prioriteitsregels alleen is men er nog niet. Zoals het voorbeeld $u := \text{LEES} / \text{LEES}$ laat zien kan de uitkomst nog afhangen van de volgorde waarin men de waarden van de primitiva bepaalt: als men eerst de eerste LEES uitvoert en daarna de tweede dan gebeurt er wat anders dan wanneer men eerst de tweede en daarna pas de eerste LEES uitvoert. Iets soortgelijks kan zich voordoen wanneer men FUN-roepen hanteert die de waarden der meegegeven [aktuele]n veranderen.

Afspraak

De primitiva in een [uitdr] worden van links naar rechts geëvalueerd. Bij een RIJ-reservering wordt eerst de [uitdr] voor de ondergrens en daarna die voor de bovengrens geëvalueerd.

Bij een [vergelijking] wordt eerst de eerste [uitdr] geëvalueerd, dan de tweede, dan (als er nog meer zijn, nl. bij het gebruik van EN en OF) de derde, etc.

10.6 Waarde van aMb

De waarde van aMb zal over het algemeen niet exact zijn. Voor niet te grote gehele waarden van a en niet-negatieve gehele waarden van b zal de uitkomst echter wel exact zijn.

11. Toekenning

Een toekenning (zie [30]) heeft de gedaante [variabele]:=[uitdr]. Bij executie wordt eerst het adres van de variabele in het linkerlid bepaald (het doeladres), daarna wordt de waarde van de uitdrukking in het rechterlid bepaald. Deze laatste waarde wordt opgeborgen in de door het doeladres aangewezen GP.

Voorbeeld

een := twee + drie

Het adres van de GP "een" wordt klaargezet als doeladres. De waarde van "twee" wordt opgehaald, de waarde van "drie" wordt daarbij opgeteld. Deze waarde wordt de nieuwe waarde van "een".

Voorbeeld

r(i+j×k) := a + b × WRTL(xM2 + yM2)

Eerst wordt de index-expressie i+j×k uitgerekend, de opgeleverde waarde wordt afgerond, dit is de index;

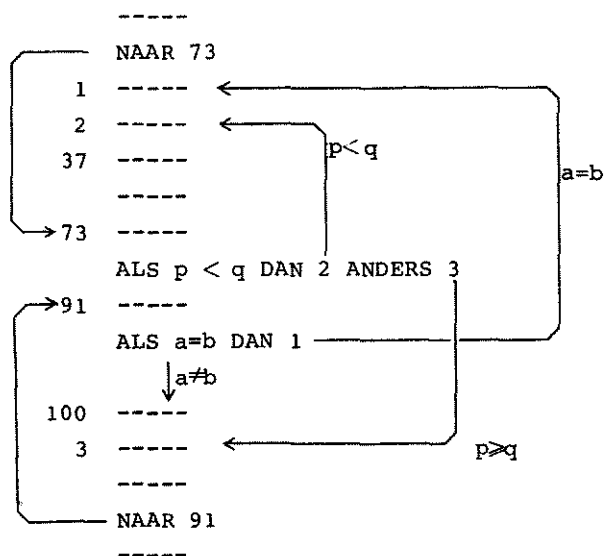
de index wordt op de grenzen uit de descriptor van "r" gecontroleerd, uit de index en het begin-adres van de rij wordt nu het adres berekend, het aldus berekende doeladres wordt klaargezet.

De waarde van het rechterlid wordt berekend uit de gegeven formule.

Deze waarde wordt opgeborgen in de GP waarvan het adres was klaargezet.

12. Sprong-opdrachten

Sprong-opdrachten dienen om bij executie een andere dan de tekstueel volgende aktie aan te wijzen als de vervolgens te executeren aktie. Een en ander blijkt voldoende uit onderstaand voorbeeld:



Men ziet dat de opdracht "ALS a=b DAN 1" hier hetzelfde doet als de opdracht "ALS a=b DAN 1 ANDERS 100" gedaan zou hebben.

Derhalve: bij een sprongopdracht wordt eerst de vergelijking achter ALS uitgewerkt. Zo'n vergelijking is WAAR of ONWAAR. Zie voor de betekenis van EN en OF onderstaand tabelletje.

De DAN-sprong wordt gedaan als de vergelijking WAAR is. Als de vergelijking ONWAAR is en er staat een ANDERS-sprong geschreven, dan wordt deze uitgevoerd, anders is de tekstueel volgende actie de vervolgens geëxecuteerde.

We brengen nog in herinnering (zie C7.4) dat wel naar allerlei acties (dus o.a. naar een RIJ-reservering) gesprongen mag worden maar niet naar START of een SUB- of FUN-beschrijving.

P	q	p EN q	p OF q
W	W	W	W
W	O	O	W
O	W	O	W
O	O	O	O

"W" is "WAAR"

"O" is "ONWAAR"

Voor een belangrijk neveneffect van sprongopdrachten zie 13 (het ongereserveerd maken van rijen).

13. RIJ-reservering

De executie van een RIJ-reservering betekent dat de waarden van onder en bovengrens worden uitgerekend, afgerond, en in de descriptor (zie 6) worden ingevuld. De aldus verkregen waarde van de bovengrens mag niet kleiner zijn dan die van de ondergrens, anders volgt een foutmelding. Vervolgens wordt in het geheugen van de computer het verlangde aantal GP's gereserveerd:

m := LEES

n := LEES

RIJ(m:n)r ; in orde als $m \leq n$, anders FOUT-melding, er wordt ruimte gereserveerd voor
; n - m + 1 GP's

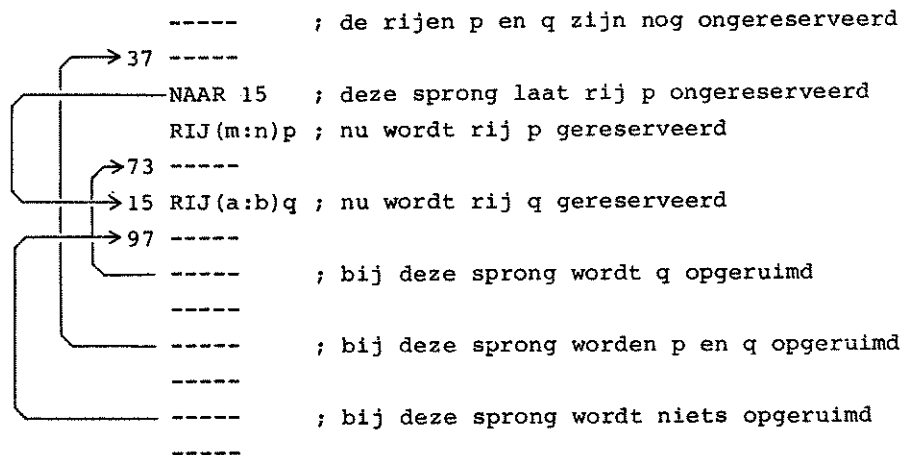
Men zie in dit verband nog de opmerkingen over volgorde in 10.6.

In plaats van RIJ(m:n)r mag men ook schrijven RIJSYM(m:n)r. Het enige verschil is dat men dan in de elementen van de rij uitsluitend gehele getallen tussen 0 en 99 (0 en 99 inbegrepen) mag onderbrengen. Deze faciliteit kan nuttig zijn bij het werken met zeer lange rijen kleine getallen, omdat de computer dergelijke RIJSYM's zuiniger in het geheugen kan opbergen dan gewone RIJen. Als men een waarde wil toekennen aan een element van een RIJSYM dan wordt deze waarde eerst afgerond. Als de afgeronde waarde niet een der getallen 0 t/m 99 is volgt een foutmelding, en wordt de executie gestaakt.

In verband met sprong-opdrachten is het volgende van veel belang:

- Voordat een programma wordt geëxecuteerd wordt ervoor gezorgd dat in de descriptor van a l l e rijen is aangetekend, dat deze "ongereserveerd" zijn.
- Wanneer men door een of andere actie een waarde wil toekennen aan of tevoorschijn halen uit een "ongereserveerde" rij, volgt o p d a t m o m e n t tijdens executie een fout-melding.
- Bij een "terugspringende" sprongopdracht (dat is een sprongopdracht die verwijst naar een plaats eerder in de programmatekst dan waar de sprongopdracht zelf staat) worden alle rijen, waarvan de reservering voorkomt in het stuk programmatekst waarover werd teruggesprongen, "ongereserveerd" verklaard en opgeruimd, d.w.z. de waarden van de elementen gaan verloren, en komen bij een later opnieuw reserveren van deze RIJ niet weer terug. (N.B. Als men "vooruit" springt over een RIJ-reservering dan was deze rij op grond van regels a en c al ongereserveerd vóór de sprong, en is dat nog steeds na de sprong omdat de betreffende reservering niet is uitgevoerd).

Voorbeeld



14. Cyclus

Volgens de syntax-regel [60] heeft een cyclus de gedaante

```

MET [stap], i := [start], [stop]
[programmeerdeel]n
[nummer] HERHAAL
  
```

(uiteraard mag bij het gebruik van de cyclus in een programmatekst ook voor MET een label staan, terwijl in [programmeerdeel] het mogen plaatsen van een label al besloten ligt, zie [21], [23] en [71]).

Hierin is [stap], [start] en [stop] ieder voor zich een [uitdr], dat wil dus zeggen: mogelijk een gecompliceerde uitdrukking met veel operanden, maar in de praktijk waarschijnlijk meestal niet gecompliceerder dan een [getal] of een heel simpele [formule]. Verder is "i" een [GP].

Losjes gezegd is de betekenis van de [cyclus] dat [programmeerdeel]ⁿ een aantal malen wordt uitgevoerd met telkens een andere waarde van i, te beginnen met de waarde van [start], oplopend met de waarde van [stap] en eindigend bij de waarde van [stop].

Voorbeelden

MET 2, i := 1, 7	}	[programmeerdeel] ⁿ wordt uitgevoerd met achtereenvolgens i = 1,3,5,7
MET 2, i := 1, 8		
MET 2, i := 1, 1		[programmeerdeel] ⁿ wordt uitgevoerd met i = 1
MET 2, i := 1, 0		[programmeerdeel] ⁿ wordt helemaal niet uitgevoerd
MET-2, i := 3, -3	}	[programmeerdeel] ⁿ wordt uitgevoerd met achtereenvolgens i = 3,1,-1,-3
MET-2, i := 3, -4		
MET-2, i := 3, 4		[programmeerdeel] ⁿ wordt helemaal niet uitgevoerd

Als de cyclus verlaten wordt heeft i de waarde waarmee de cyclus voor het laatst doorlopen is, resp. de waarde van [start] als [programmeerdeel]ⁿ helemaal niet uitgevoerd is.

Zelfs bij deze eenvoudige voorbeelden moet nog een voorbehoud worden gemaakt: als men namelijk binnen [programmeerdeel]ⁿ de waarde van i verandert kan er best iets heel anders gebeuren.

Daarom zullen we nu precies zeggen wat er in een cyclus gebeurt. De cyclus is gelijkwaardig met het volgende stuk ECOL-programma. Hierin zijn de onderstreepte namen teller, stap en stop hulpvariabelen die door de compiler worden toegevoegd en waar de programmeur niet bij kan komen, en evenzo zijn label1, label2 en label3 labels die de programmeur niet kan bereiken.

Met dit stukje programma worden de kwadraten van de getallen 0 t/m 99 afgedrukt, telkens met tien op een regel. De vraag welke HERHAAL bij welke MET hoort is eenvoudig te beantwoorden: als twee MET's op elkaar volgen zonder HERHAAL ertussen komt de HERHAAL van de tweede MET vóór de HERHAAL van de eerste MET (vergelijk de haakjes structuur bij [uitdr]ukkingen).

In een cyclus mogen ook RIJ-reserveringen en (alhoewel dit geen bijzondere effecten heeft) ook SUB-beschrijvingen voorkomen. Dit is echter verboden bij het gebruik in een FUN-beschrijving, omdat daar geen RIJ-reserveringen en SUB-beschrijvingen mogen voorkomen.

15. Lege opdracht

Volgens [23] mogen in een ECOL tekst ook regels voorkomen waarop behalve eventueel een label en kommentaar verder niets staat, zulks ter bevordering van de leesbaarheid voor de menselijke lezer. Een dergelijke regel wordt bij executie gewoon overgeslagen, ook als men er via een sprong op terecht komt. Het is dus net of er op zo'n regel een opdracht staat die niets doet, een lege opdracht.

16. Invoer

```
16.1      [variabele] := LEES
          [variabele] := LEESYM
```

Met LEES wordt het eerstvolgende getal van een sequentieel extern medium gelezen, bijv. een ponsband of schrapkaart, en dit getal wordt als waarde toegekend aan de variabele in het linkerlid (de term sequentieel staat tegenover adresseerbaar; men kan niet zeggen: "ik wil de zoveelste hebben" maar slechts "geef mij de volgende"). Een getal op het externe medium moet een gedaante hebben als in syntax [38], eventueel voorafgegaan door + of - en ook hier geldt dat spaties geen betekenis hebben. LEES leest dan ook op het externe medium symbool na symbool totdat een cijfer verschijnt of een + of - gevolgd door een cijfer; hiermee is dan een getal begonnen, en dit getal wordt geacht op te houden zodra een symbool verschijnt dat niet meer tot het getal kan behoren, dus een symbool dat geen punt of cijfer is, of ook het symbool punt zelf als er in het getal al een punt is voorgekomen. Bij dit alles worden spaties steeds veronachtzaamd.

Het eerste symbool dat niet meer tot het getal behoort is tevens het eerste symbool dat door een volgende LEES of LEESYM gelezen wordt.

Als LEES tijdens het lezen van een getal bij het einde van het medium komt, is hiermee het getal ook afgelopen. Als LEES op het externe medium geen getal meer aantreft volgt een foutmelding, alsmede beëindiging der executie van het programma.

Met LEESYM wordt het eerstvolgende symbool van een sequentieel extern medium gelezen, en de interne waarde van dit symbool wordt aan de variabele in het linkerlid toegekend. De interne waarde van een extern symbool is een geheel getal. De relatie is in onderstaande tabel vastgelegd. Als LEESYM een ander symbool ontmoet dan de onder 0 t/m 91 genoemde, wordt de waarde 98 afgeleverd (zie os = onbekend symbool). Als het externe medium uitgeput is levert LEESYM de waarde 99 af (zie et = einde tekst).

Als na LEESYM een LEES wordt uitgevoerd is het eerste symbool dat LEES leest het laatste dat LEESYM gelezen heeft. Hierdoor is het mogelijk met LEESYM te speuren naar het begin van een nieuw getal en dan toch nog met LEES het hele getal te lezen.

TABEL van symbolen en hun interne waarde:

cijfers		letters								overige symbolen									
sym get		sym get		sym get		sym get		sym get		sym get		sym get		sym get					
0	0	a	10	k	20	u	30	A	40	K	50	U	60	.	70	+	80	sp	90
1	1	b	11	l	21	v	31	B	41	L	51	V	61	,	71	-	81	nr	91
2	2	c	12	m	22	w	32	C	42	M	52	W	62	:	72	x	82		
3	3	d	13	n	23	x	33	D	43	N	53	X	63	;	73	/	83		
4	4	e	14	o	24	y	34	E	44	O	54	Y	64	?	74	=	84		
5	5	f	15	p	25	z	35	F	45	P	55	Z	65	!	75	<	85		
6	6	g	16	q	26			G	46	Q	56			'	76	>	86		
7	7	h	17	r	27			H	47	R	57			"	77	≤	87		
8	8	i	18	s	28			I	48	S	58			(	78	≥	88	os	98
9	9	j	19	t	29			J	49	T	59			)	79	≠	89	et	99

Met sp en nr bedoelen we 'spatie' en 'overgang op nieuwe regel', met et bedoelen we 'einde tekst' (om aan te geven dat een tekst afgelopen is), met os een 'overig symbool' of 'onbekend symbool'.

Voorbeeld 16

Het externe medium bevat

+.- 5 . A+.5 7.23 . -17

en wordt gelezen met

```

a:= LEES      ; leest tot A, -5 naar a
b:= LEES      ; begint bij A en leest tot vijfde punt; 57.23 naar b
c:= LEESY     ; leest vijfde punt; 70 naar c
d:= LEESY     ; leest spatie; 90 naar d
e:= LEESY     ; leest -; 81 naar e
f:= LEES      ; begint bij - en leest tot einde; -17 naar f
g:= LEESY     ; medium uitgeput; 99 naar g

```

Tenslotte: men mag LEES en LEESY ook in [uitdr]ukkingen gebruiken (zie 18).

16.2

$$\begin{array}{ccc} & h & k \\ & \downarrow & \downarrow \\ \text{[GP-rij]} & := \text{BESTAND}(\text{[uitdr]}, \text{[uitdr]}, \text{[uitdr]}) \end{array}$$

De eerste parameter "[uitdr]" is altijd het bestand-nummer (mogelijk is op sommige computers hier alleen maar een [getal] toegestaan i.p.v. een willekeurige [uitdr]).

De opdracht kopieert het stuk h t/m k van het aangewezen bestand in de rij met de naam [GP-rij].

Het kopiëren eindigt zodra of de rij, of het aangegeven stuk bestand uitgeput is.

Alle [uitdr] worden weer eerst afgerond.

16.3 [GP-rij] := BESTAND([uitdr])

doet hetzelfde als:

[GP-rij] := BESTAND([uitdr], 1, totale lengte van het bestand)

17. Uitvoer

17.1

$$\begin{array}{ccc} & m & n \\ & \downarrow & \downarrow \\ \text{SCHRIJF}(\text{[uitdr]}, \text{[uitdr]}) & := & \text{[uitdr]} \end{array}$$

Het uitvoerapparaat (in het algemeen een regeldrukker) drukt de waarde van de [uitdr] rechts van het wordt-teken in decimale voorstelling af met teken (een plusteken wordt mogelijk vervangen door spatie), m cijfers voor de decimale punt en n cijfers erachter, in totaal dus m+n+2 posities. Bij

het gedeelte voor de punt worden niet-significante nullen onderdrukt. Als $n=0$ worden evenwel geen decimale punt met cijfers erachter afgedrukt, dus slechts $m+1$ posities. De [uitdr] voor m en n worden weer afgerond.

Wanneer het gedeelte vóór de decimale punt meer cijfers nodig heeft dan door m is aangegeven, of wanneer m te groot is (varieert van machine tot machine) dan wordt het getal afgedrukt volgens SCHRIJM.

17.2 SCHRIJF := [uitdr]

Als de waarde van [uitdr] geheel is doet deze opdracht hetzelfde als $SCHRIJF(6,0) := [uitdr]$, in het andere geval hetzelfde als $SCHRIJF(6,4) := [uitdr]$.

In beide gevallen worden zoveel spaties toegevoegd dat op het uitvoerapparaat evenveel posities worden bezet als met SCHRIJM.

17.3 SCHRIJM := [uitdr]

Het uitvoer-apparaat drukt de afgeronde waarde rechts van het ":" teken in decimale voorstelling af als een mantisse tussen 0 en 1 gevolgd door de exponent van de macht van 10 waarmee dit getal vermenigvuldigd moet worden om de waarde van de meegegeven [uitdr] voor te stellen. Zowel mantisse als exponent worden voorafgegaan door een voorteken.

Het aantal decimalen van mantisse en exponent is afhankelijk van de machine waarop het programma wordt verwerkt.

Voorbeeld

SCHRIJF := $\pi \times 10^5$	⇒	314159.2654
SCHRIJF(3,5) := $\pi \times 10^5$	⇒	314.15927
SCHRIJF := $-\pi \times 10^5$	⇒	- 3141.5927
SCHRIJM := $\pi \times 10^5$	⇒	+0.3141592653589E+6
SCHRIJF(5,5) := $\pi \times 10^5$	⇒	+0.3141592653589E+6
SCHRIJF := -12	⇒	- 12

17.4 SCHRIJFSYM := [uitdr]

Er wordt gekeken of de op een geheel getal afgeronde waarde van [uitdr] hoogstens 91 is en voorkomt in de tabel van symbolen (zie 16.1). Zo ja, dan wordt het symbool met die waarde afgedrukt; zo nee, dan wordt een bijzonder symbool afgedrukt. Wat er op papier verschijnt als het uitvoer-apparaat een symbool moet afdrukken waartoe het niet in staat is (bijv. een kleine letter als het alleen hoofdletters kent) kan van machine tot machine verschillen.

Voorbeeld

SCHRIJFSYM := 74 drukt af ?

17.5 TEKST := "[sym]ⁿ⁺¹"

Afgedrukt wordt de tekst tussen de aanhalingstekens; in deze tekst mag echter geen aanhalingsteken voorkomen en geen [⌋].

17.6 SPATIE([uitdr])

Het uitvoer-apparaat geeft net zoveel spaties als door de afgeronde waarde van [uitdr] wordt aangegeven; als deze afgeronde waarde ≤ 0 is, gebeurt er niets.

17.7 SPATIE

doet hetzelfde als de opdracht:

SPATIE(1)

17.8 NR([uitdr])

Het uitvoer-apparaat schuift het papier evenveel regels omhoog als door de afgeronde waarde van [uitdr] wordt aangegeven; als deze afgeronde waarde ≤ 0 is, gebeurt er niets met het papier. Na het omhoogschuiven, maar ook bij "NR(0)" begint het afdrukken weer vooraan de regel (dit vooraan laten

beginnen bij NR(0) kan bij sommige machines op moeilijkheden stuiten).

17.9 NR

doet hetzelfde als de opdracht

NR(1)

17.10 NP

Deze opdracht laat op het uitvoerapparaat een nieuwe pagina verschijnen.

17.11

h k
↓ ↓
BESTAND([uitdr] , [uitdr] , [uitdr]) := [GP-rij]

De eerste parameter "[uitdr]" is altijd het bestand-nummer (zie opmerking in 16.2).

De opdracht kopieert de rij met de naam [GP-rij] in het stuk h t/m k van het aangewezen bestand. Het kopiëren eindigt zodra of de rij, of het aangegeven stuk bestand uitgeput is.

17.12 BESTAND([uitdr]) := [GP-rij]

doet hetzelfde als de opdracht:

BESTAND([uitdr] , 1, totale lengte van het bestand) := [GP-rij]

17.13 Opmerkingen uitvoer

De wijze waarop de computer reageert op situaties zoals:

- men wil meer symbolen op een regel hebben dan mogelijk is bij een bepaald uitvoer-apparaat,
- men wil een symbool afdrukken dat er niet is op het uitvoerapparaat

kan van machine tot machine verschillen, evenals de voorstelling van allerlei symbolen en tekens (bijv. "*" in plaats van "x", "<=" in plaats van "≤", "10" in plaats van "E" in een SCHRIJF-
getal).

Men bedenke hierbij ook, dat vele computers regeldrukkers van vaak uiteenlopend type hebben, en het lang niet zeker is dat de ECOL-uitvoer altijd over hetzelfde type regeldrukker loopt.

18. Standaard-functies

De volgende standaard-functies worden in iedere ECOL-implementatie verondersteld: ("u" staat hier steeds voor een [uitdr], tenzij anders vermeld):

G(u)	levert het gehele deel van u af:	G(2.7) is +2 G(-2.3) is -3
ABS(u)	levert de absolute waarde van u af	
WRTL(u)	levert $\sqrt{u}$ af als $u \geq 0$; anders foutmelding	
SQRT(u)	doet hetzelfde als WRTL(u)	
SIN(u)	levert de waarde van de sinus van u (in radialen) af	
COS(u)	levert de waarde van de cosinus van u (in radialen) af	
ARCTAN(u)	levert de waarde van de hoek (in radialen) af die gelegen is tussen $-\pi/2$ en $+\pi/2$ en waarvan de tangens u is	
LN(u)	levert $e \log(u)$ af als $u > 0$; anders foutmelding	
LOG(u)	levert $10 \log(u)$ af als $u > 0$; anders foutmelding	
EXP(u)	levert de waarde van e tot-de-macht u af	
GOK(u1,u2)	levert (pseudo) aselechte getallen af, t.w. met ongeveer gelijke waarschijnlijkheid een der gehele getallen tussen de afgeronde waarden van u1 en u2, deze afgeronde waarden inbegrepen. Dus GOK(1,6) en GOK(6,1) werken als dobbelsteen	

GOKC levert (pseudo) aselechte getallen af, t.w. met ongeveer gelijke waarschijnlijkheid een der getallen tussen 0 en 1, 0 wel en 1 niet inbegrepen

ONDIN([GP-rij]) levert de ONDerINdex af van de rij met naam [GP-rij]

BOVIN([GP-rij]) " " BOVenINdex af van " " " [GP-rij]

LEES leest het volgende getal van de band en levert daarvan de waarde af net zoals in de invoer-opdracht [variabele] := LEES. Men mag dus ook schrijven bijv. "y := LEES + 1"

LEESSYM idem maar nu voor het volgende symbool op de band of kaart

SKIP doet net zo vaak LEESSYM totdat [↵] is gepasseerd

PS (positie schrijfkop) levert het aantal symbolen af dat door het uitvoerapparaat op de huidige regel is afgedrukt

PP (positie papier) levert het aantal voltooide regels op de huidige pagina van het uitvoer-apparaat

STOP de executie van dit programma wordt beëindigd

V. SUB EN FUN

19. Inleiding

Het is bij het schrijven van programma's niet ongebruikelijk dat men in verschillende delen van zijn programma gelijke of bijna gelijke reeksen opdrachten nodig heeft. Het SUB en FUN mechanisme maakt het nu mogelijk zo'n reeks opdrachten eenmaal op te schrijven, waarna ze van verschillende plaatsen door één simpele opdracht kunnen worden geactiveerd. Men zie bijv. de standaardfuncties (zie 18), die eenmaal ergens in het geheugen van de computer uitgeprogrammeerd staan, en die men in zijn programma overal heel simpel kan aanroepen. De SUB is een heel eenvoudig mechanisme, dat dan ook niet zo erg flexibel is, de FUN opent veel verder gaande mogelijkheden.

20. SUB-beschrijving

Volgens syntax-regel [71] is een [SUB-beschrijving] :

```
[nummer] SUB[ SUB-naam] [
[opdracht]n
[nummer] END [
```

In een SUB-beschrijving mogen dus geen RIJ-reserveringen en ook geen SUB-beschrijvingen voorkomen. Wel zijn alle [opdrachten] , inclusief de [cyclus] en de [SUB-roep] , toegestaan.

Omdat een SUB-beschrijving deel uitmaakt van het hoofdprogramma, gelden in het bijzonder de in 7 en 8 behandelde context-regels.

Men noemt een SUB-beschrijving ook wel een subroutine.

Voorbeeld van een SUB-beschrijving

```
100 SUB skip tot nul                    ; deze subroutine leest de getalband tot en met het eerst-
                                         ; volgende getal 0

101 getal := LEES

102 ALS getal ≠ 0 DAN 101

103 END                                  ; de subroutine wordt verlaten via de achterdeur, zoals het be-
                                         ; hoort, zie 8.
```

Deze SUB-beschrijving mag overal in het hoofdprogramma staan mits buiten een SUB-beschrijving, en ook overal in het hoofdprogramma worden aangeroepen (dus wel in een SUB-beschrijving, niet in een FUN-beschrijving).

Voorbeeld

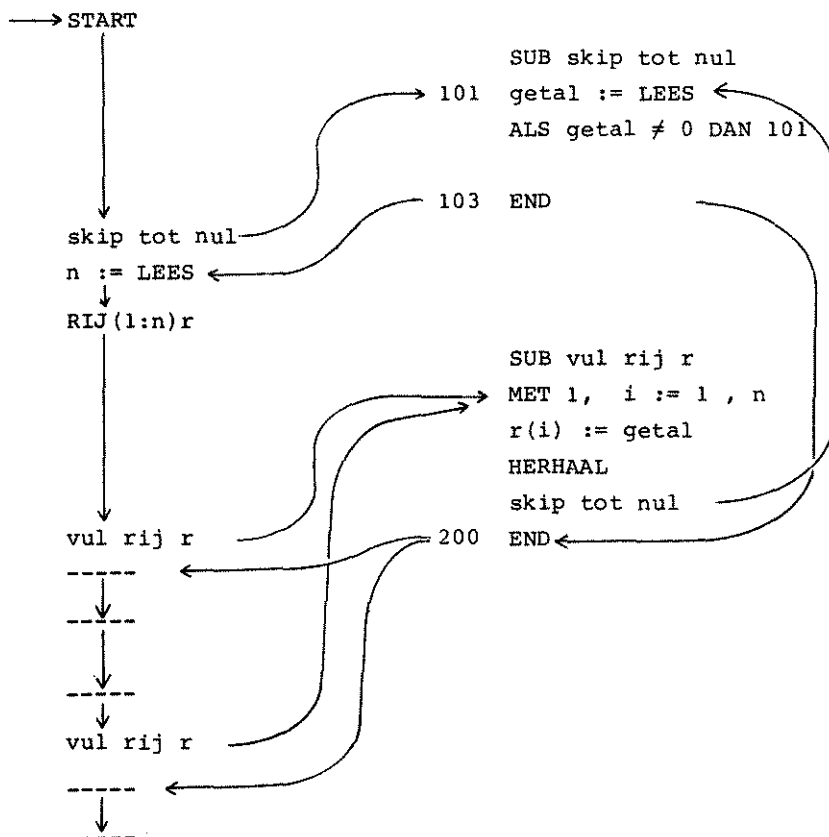
```
SUB vul rij r          ; deze subroutine vult de rij "r" met gelezen getallen
MET 1, i := 1, n
r(i) := LEES
HERHAAL
skip tot nul          ; als de rij gevuld is, worden alle getallen tot en met 0 geskipt.
END
```

Deze SUB-beschrijving moet tekstueel volgen op de reservering van de rij "r" (zie C7.3).

21. SUB-roep

De executie van een SUB-roep betekent, dat het programma van de aangeroepen SUB-beschrijving wordt doorlopen. Zodra daarbij "END" wordt bereikt, wordt teruggekeerd naar de actie die tekstueel volgt op de SUB-roep. In dit verband wordt ook END als actie beschouwd (zie in onderstaand voorbeeld de pijl van 103 naar 200).

Een en ander wordt geadstrueerd aan onderstaand stuk programma, waarin de voorbeelden van 20 worden aangeroepen:



N.B. Een SUB-roep lijkt wel wat op een sprong. Men mag echter nooit met een sprongopdracht naar een SUB-beschrijving springen (zie 8). Ook worden rijen, die gereserveerd worden tussen de plaats van SUB-roep en SUB-beschrijving, door de SUB-roep niet ongereserveerd gemaakt (vgl. 13).

22. Informele inleiding tot het FUN mechanisme

De SUB voorziet in de behoefte wanneer men op verschillende plaatsen van het programma precies dezelfde opdrachtenreeks nodig heeft. Heel vaak echter heeft men op verschillende plaatsen van het programma slechts ongeveer dezelfde opdrachten nodig, opdrachten die wel hetzelfde rekenschema representeren, maar dat met telkens andere variabelen uitvoeren. Hierin voorziet de FUN.

In de FUN-beschrijving geeft men het rekenschema weer voor fictieve grootheden (hier "formelen" genoemd), terwijl men dan bij de aanroep zegt wat de werkelijke grootheden zijn waarop geopereerd moet worden (hier "aktuellen" genoemd). Als aktuelen kan men meegeven uitdrukkingen (dus o.a.

variabelen) maar ook RIJ-namen. In de FUN-beschrijving moet men aangeven wat voor soort aktuelen men wenst mee te geven.

Voorbeeld van FUN-beschrijving

```
FUN som(h,k,RIJ a)
  hulp := 0
  MET l, i:=h, k
  hulp := hulp + a(i)
  HERHAAL
  FUN := hulp
```

Deze FUN bepaalt voor een meegegeven RIJ, die we hier maar zolang a noemen ("formele" naam) de som van het h-de tot en met het k-de element (h en k zijn ook formelen). De aanduiding RIJ vóór a geeft aan dat we bij de aanroep als derde parameter de naam van een rij mee moeten geven. Het ontbreken van deze aanduiding bij h en k geeft aan dat men bij de aanroep op de eerste en tweede plaats een uitdrukking moet zetten. Nu een

Voorbeeld van aanroep van deze FUN vanuit het hoofdprogramma

```
-----
-----
RIJ(1:500)m
-----
-----
p := som(6, 34, m)
q := som(3, m(6), m)
```

Met de eerste aanroep worden de elementen 6 t/m 34 van de rij m opgeteld, en de som wordt aan p als waarde toegekend. Bij de tweede aanroep wordt de som van de elementen 3 t/m m(6) aan q toegekend. Dat m, een rijnaam, ook nog eens op de plaats van de tweede parameter voorkomt, terwijl de tweede parameterplaats in de FUN-beschrijving niet met RIJ is gemerkt, is niet onjuist: immers de tweede aktuele luidt m(6), dat is dus gewoon een variabele, dus een uitdrukking.

Een heel belangrijke, en ook doorgaans prettige, eigenschap van de FUN is dat alle er in optredende labels en namen (behalve de namen van andere FUN's) lokaal zijn, d.w.z. niets te maken hebben met identieke labels en namen die mogelijk in andere FUN's of in het hoofdprogramma gebruikt worden. En in een bepaalde FUN-beschrijving hoeft men met de namen van andere FUN's alleen nog maar rekening te houden voor zover men die andere FUN's vanuit deze FUN-beschrijving wenst aan te roepen. De namen van FUN's die men niet wenst aan te roepen mag men rustig als (lokale) GP's of formelen gebruiken. Bij het schrijven van een FUN kan men dus de te gebruiken labels en namen (behalve namen van aan te roepen FUN's) vrij kiezen, ongeacht welke namen in andere FUN-beschrijvingen of in het hoofdprogramma gebruikt worden. Dit maakt het o.a. mogelijk voor vaak voorkomende ingewikkelde processen (bijv. het oplossen van stelsels vergelijkingen) FUN-beschrijvingen te maken die men aan elk willekeurig programma kan toevoegen.

23. FUN-beschrijving

Volgens [86] t/m [89] begint een FUN-beschrijving met FUN gevolgd door de naam die men aan de FUN wenst te geven, gevolgd door een rijtje formelen, minstens één, van elkaar gescheiden door komma's dit rijtje gevat in haakjes. Elke formele heeft de gedaante [GP] (d.w.z. [naam]) of RIJ [GP-rij] (d.w.z. het woordje RIJ gevolgd door een [naam]).

Na deze "kop" van de FUN-beschrijving komen [opdrachten], d.w.z. dat RIJ-reserveringen en SUB-beschrijvingen niet zijn toegestaan. Maar ook zijn SUB-roepen niet toegestaan, zoals aanstonds zal blijken.

De FUN-beschrijving eindigt tenslotte met FUN := [uitdr] (en niet met END, zoals de SUB-beschrijving).

Een zelfgeschreven FUN moet dus, in afwijking van de standaardfuncties, minstens één parameter hebben, ook al heeft men die eigenlijk helemaal niet nodig. Men is evenwel niet verplicht deze

parameter in de rest van de FUN-beschrijving ook te gebruiken. Overigens zou men met parameterloze, procedures ook niet erg veel kunnen aanvangen, enkele bijzondere gevallen daargelaten. Evenzo moet een FUN-beschrijving eindigen met FUN :---, ook al is men helemaal niet van plan van de aldus toegekende waarde gebruik te maken, bijv. doordat de FUN-roepen van deze FUN nooit in een [uitdr] maar altijd als [opdracht] plaats hebben (zie [23] en [37]).

De verschillende [FUN-beschrijvingen] zijn dermate onafhankelijk van elkaar dat men voor elke [FUN-beschrijving] apart bij het in tekstuele volgorde lezen weer zekerheid heeft over de status van een [naam] bij het eerste optreden ervan.

Context-regels voor het funktiedeel

Voor elke [FUN-beschrijving] afzonderlijk geldt:

- C23.1 Een en dezelfde [naam] heeft overal in deze [FUN-beschrijving] dezelfde betekenis.
- C23.2 Een [naam] is een [GP-rij] als hij in deze [FUN-beschrijving] optreedt op de plaats van [formele] voorafgegaan door de aanduiding RIJ (zie [86] en [89]).
- C23.3 Een [naam] is een [FUN-naam] als hij in deze [FUN-beschrijving] voorkomt gevolgd door een openingshaakje (spaties spelen geen rol), en niet valt onder C23.2.
- C23.4 Alle overige namen zijn [GP]'s voor deze [FUN-beschrijving].
- C23.5 Alle [labels] in een [FUN-beschrijving] moeten van elkaar verschillen.
- C23.6 Alle [namen] in deze [FUN-beschrijving] die geen [FUN-naam] zijn, en alle [labels] hebben niets te maken met eventuele identieke [namen] en [labels] elders in het programma.

Voor het funktiedeel als geheel geldt:

- C23.7 Elke [naam] die in een of meer der [FUN-beschrijvingen] als [FUN-naam] optreedt moet ook precies éénmaal optreden op de plaats van [FUN-naam] in een [FUN-beschrijving].

Uit deze context-regels volgt nog dat SUB-roepen vanuit een FUN-beschrijving onmogelijk zijn, want SUB-namen zijn namen in het hoofdprogramma.

Voorbeeld

```

FUN f(x)      ; Hiermee is "f" voor het gehele programma, inclusief het funktiedeel, een
               ; [FUN-naam], "x" is, alleen voor deze FUN-beschrijving, een formele [GP]
y := x        ; Hiermee is "y", echter alleen voor deze FUN-beschrijving, een zg. "lokale"
               ; [GP]
1  x := r(y)   ; "r" kan onmogelijk de naam zijn van een [GP-rij] want dan zou "r" als zo-
               ; danig formeel moeten zijn. De compiler vertaalt "r" nu als de naam van een
               ; FUN met 1 formele [GP], waarvan de beschrijving nog moet volgen
               ; De label "1" is ook "lokaal", dat wil zeggen: geldt alleen voor deze FUN-
               ; beschrijving
FUN := x+y     ; Hiermee wordt deze FUN-beschrijving afgesloten
               ; De betekenis van de namen "x" en "y" en van de label "1" gaan nu verloren,
               ; n i e t echter de betekenis van de namen "f" en "r" als FUN-namen
               ; De compiler verwacht nog de beschrijving van "r".
FUN r(x,RIJp); FOUT! "r" mocht alleen maar 1 formele [GP] hebben
-----
-----
FUN := -----
FUN r(x)      ; Dat verwacht de compiler (zie overigens C23.7)
-----
-----
FUN :=-----

```

24. FUN-roep

De executie van een FUN-roep betekent, dat het programma van de aangeroepen FUN-beschrijving wordt doorlopen. Zodra daarbij "FUN :=[uitdr]" wordt bereikt, wordt teruggekeerd naar het punt vanwaar de funktie werd aangeroepen. Wanneer dit plaats vond in een [uitdr] , is de waarde van de FUN-roep, de aan "FUN" toegekende waarde; wanneer de funktie als [opdracht] werd geroepen, speelt deze waarde verder geen rol (zie voorbeeld 24.2).

In een FUN-roep moet op de plaats van een formele [GP] als [aktuele] een [uitdr] worden meegegeven en op de plaats van een formele [GP-rij] een (aktuele) [GP-rij] (niet voorafgegaan door de aanduiding RIJ). Deze rij dient ten tijde van de aanroep gereserveerd te zijn.

Als de aktuele [uitdr] een [variabele] is, wordt bij executie het adres van die [variabele] aan de funktie doorgegeven, anders wordt die [uitdr] uitgerekend en in een voor de programmeur onbereikbare hulp-GP gestopt waarvan bij executie ook het adres aan de funktie wordt meegegeven.

De waarden der bedoelde [uitdr]ukkingen en de adressen der [variabelen] worden weer bepaald in de tekstuele volgorde der parameters (vgl. 10.6).

Omdat bij executie het adres van de aktuele wordt meegegeven opereert de FUN ook echt op die aktuele. Wordt in een FUN-beschrijving een waarde toegekend aan een formele GP, dan wordt bij aanroep aan de aktuele GP een waarde toegekend. Zo kan de FUN, ondanks context-regel C23.6 toch via de parameters waarden van GP's in het hoofdprogramma veranderen.

Evenzo wordt bij een aktuele [GP-rij] de descriptor van de rij meegegeven, zodat weer op de echte rij-elementen wordt geopereerd.

Voorbeeld 24.1

```
FUN f(x,y)
  x := 3
  FUN := y
```

Met een aanroep f(a,b) krijgt a de waarde 3. Met een aanroep f(a+b,c) of f(17,q) krijgt de hulp-GP, waar de waarde van a+b resp 17 ingestopt was, de waarde 3, maar met a, b of overige getallen 17 in het programma gebeurt niets. Interessant is nog de aanroep f(i,d(i)). Als bij aanroep i bijv. de waarde 5 heeft, krijgt FUN de waarde d(5), en niet d(3), zoals men misschien zou denken (omdat i door FUN de waarde 3 krijgt). Immers bij aanroep wordt het adres van de parameters meegegeven, dus het adres van d(5).

Als de FUN-beschrijving daarentegen geluid had

```
FUN g(x,RIJ y)
  x := 3
  FUN := y(x)
```

dan zou bij aanroep met g(i,d) (d een op dat moment gereserveerde rij) FUN de waarde d(3) hebben gekregen.

Voorbeeld 24.2

```
FUN max(index,RIJ r)           ; deze funktie levert het grootste getal van de rij r en kent
                                ; aan "index" de waarde toe van de index waarvoor dat maximum
                                ; wordt bereikt
h := ONDIN(r)                  ; h := onderindex van r, zie 18
k := BOVIN(r)                  ; k := bovenindex van r, zie 18
index := h
probeer := r(h)
MET 1, i := h+1, k
  volgende := r(i)
  ALS volgende ≤ probeer DAN 1
    index := i
    probeer := r(i)
1 HERHAAL
FUN := probeer
```

Als "max" wordt aangeroepen bijvoorbeeld in:

```
y := max(i, rij)
```

wordt aan "y" de waarde van het maximale element van de aktuele "rij" toegekend, terwijl de index waarvoor dit maximum wordt bereikt in "i" is te vinden.

Deze "i" uit de aanroep (een GP uit het hoofdprogramma, of een andere FUN-beschrijving!) heeft niets te maken met de "i" van de beschrijving "max".

Als "max" wordt aangeroepen als [opdracht]:

```
max(u,rij)
```

gooien we de waarde van het maximale getal van "rij" weg, maar die kunnen we altijd weer terugvinden door de opdracht:

```
grootste := rij(u)
```

Als "max" wordt aangeroepen bijvoorbeeld als in:

```
y := max(0,rij)
```

wordt de aktuele waarde 0 in een voor de programmeur onbereikbare GP gestopt. Deze onbereikbare GP gaat bij het doorlopen van "max" nu fungeren als hulp-GP voor de formele "index", maar de laatste waarde daarvan ben je wel kwijt als de funktie klaar is!

25. Recursiviteit

Funkties mogen ook "in recursie" gaan, dat wil zeggen: zichzelf (hetzij direkt of via een andere funktie) aanroepen. Bij recursief gebruik kan een funktie dus in verschillende "incarnaties" (of "diepten") worden geactiveerd. Er komt dan nog een regel bij:

C25.1 Als een funktie in recursie wordt aangeroepen, worden voor alle lokale-, niet-formele [GP]'s, nieuwe geheugenplaatsen gereserveerd, die niets te maken hebben met de geheugenplaatsen die voor die [GP]'s in enige vorige incarnatie waren gereserveerd.

Dit recursie mechanisme is een afspiegeling van en kan gebruikt worden bij rekenmachine implementatie voor recursieve processen in de wiskunde. Een heel bekend voorbeeld van zo'n recursief wiskundig proces is de algoritme van Euclides voor het bepalen van de grootste gemene deler van twee (niet noodzakelijk positieve) gehele getallen m en n. In woorden luidt deze algoritme:

```
als n = 0 dan is de ggd van m en n het getal m
als n ≠ 0 dan is de ggd van m en n hetzelfde als de ggd van n en de rest die ontstaat
bij deling van m door n.
```

Men vindt dit vrijwel letterlijk terug in onderstaande FUN-beschrijving. Alleen, omdat men in een FUN-beschrijving niet tweemaal mag zeggen FUN :=--- vindt men op regel 2 een toekenning aan m, die op dat moment toch overbodig is en als hulpvariabele dienst doet.

```
FUN ggd(m,n)
1 ALS n = 0 DAN 3
2 m := ggd(n,m-n × G(m/n)) ; m - n×G(m/n) is rest bij deling van m door n
3 FUN := ABS(m) ; we maken de ggd positief
```

VI DE ECOL-SYNTAX

In de syntax wordt min of meer vastgelegd welke teksten wel, en welke niet door de compiler als correct ECOL worden geaccepteerd. Hierbij maken we gebruik van zg. "produktie-regels", zoals [5], waarvan de betekenis geen toelichting zal behoeven, en waarbij een ieder zich wel kan voorstellen welke letters niet expliciet opgeschreven zijn.

In [8] is ruimte gelaten voor eventueel nog enkele leestekens die mogelijk niet op elk apparaat (of schrapkaart) produceerbaar zijn; de gegeven leestekens zijn de minimaal vereiste.

Vele produktieregels produceren een opeenvolging van syntactische eenheden, zoals [55]. Men moet dit zo lezen dat een [RIJ-reservering] bestaat uit het woordje RIJ, gevolgd door een openingshaakje, waarna iets komt dat door de produktieregel voor [uitdr] (zie [34]) geproduceerd wordt, waarna dubbelpunt etc.

In een aantal produktieregels komt een "exponent" n of $n+1$ voor. Bijvoorbeeld in [12], waar dit aanduidt dat een [naam] een opeenvolging is van $n+1$ [kleine letter]s. Steeds is $n = 0, 1, 2, 3, \dots$. Een [naam] bestaat dus uit minstens één kleine letter.

Evenzo betekent [opdracht]ⁿ (zie [86]) dat er n [opdracht]en na elkaar mogen komen, maar omdat n ook 0 mag zijn mag er ook helemaal niks staan. Als er meer dan één [opdracht] staat moeten deze vanzelf onder elkaar komen te staan, omdat krachtens [23] elke opdracht wordt beëindigd met [] (zie hiervoor [16]).

In enkele gevallen tenslotte is een produktieregel recursief, zoals bijv. [88]; men moet dit zo lezen:

[formelen] is [formeel] of [formeel],[formeel]
of [formeel],[formeel],[formeel] etc

Bij de recursie kunnen meerdere produktieregels betrokken zijn, zoals het geval is in [34] t/m [37].

Op vanzelfsprekende wijze worden in de hoofdstukken I t/m V van dit rapport verwijzingen naar syntactische begrippen ook in de meervoudsvorm gegeven, dus [namen],[labels], e.d.

Voor apparaten met een kleinere symbolen-voorraad dan minimaal voor ECOL vereist (bijv. geen onderscheid tussen [kleine letter] en [HOOFDLETTER]), kunnen eenvoudige conventies worden opgesteld, die het toch mogelijk maken ECOL-programma's via een dergelijk apparaat in te voeren. Deze conventies zijn echter veelal machine-, althans implementatie-gebonden en worden hier verder buiten beschouwing gelaten.

Spatieconventie

In de syntax is eenvoudigheidshalve niet tot uitdrukking gebracht dat overal in een [ECOL-programma] spaties mogen worden toegevoegd zonder dat daardoor de betekenis van het programma verandert, behalve in een [hoofdletterwoord] (= [HOOFDLETTER]ⁿ⁺¹) en tussen de aanhalingstekens achter TEKST:=. Er moet echter minstens één spatie staan wanneer twee [hoofdletterwoord]en direct op elkaar volgen.

[1] [sym]	<u>is</u>	[⤵] <u>of</u> [symbool]	
[2] [⤵]	<u>is</u>	[overgang op nieuwe regel, of einde kaart]	
[3] [symbool]	<u>is</u>	[cijfer] <u>of</u> [kleine letter] <u>of</u> [HOOFDLETTER] <u>of</u> [teken] <u>of</u> [spatie]	
[4] [cijfer]	<u>is</u>	0 <u>of</u> 1 <u>of</u> 2 <u>of</u> 3 <u>of</u> 4 <u>of</u> 5 <u>of</u> 6 <u>of</u> 7 <u>of</u> 8 <u>of</u> 9	
[5] [kleine letter]	<u>is</u>	a <u>of</u> b <u>of</u> c <u>of</u>	<u>of</u> x <u>of</u> y <u>of</u> z
[6] [HOOFDLETTER]	<u>is</u>	A <u>of</u> B <u>of</u> C <u>of</u>	<u>of</u> X <u>of</u> Y <u>of</u> Z
[7] [teken]	<u>is</u>	[leesteken] <u>of</u> [operator] <u>of</u> [vergelijker]	
[8] [leesteken]	<u>is</u>	. <u>of</u> , <u>of</u> : <u>of</u> ; <u>of</u> ? <u>of</u> ! <u>of</u> (<u>of</u>) <u>of</u>	<u>of</u> "
[9] [operator]	<u>is</u>	+ <u>of</u> - <u>of</u> × <u>of</u> / <u>of</u> M	
[10] [vergelijker]	<u>is</u>	= <u>of</u> < <u>of</u> > <u>of</u> ≤ <u>of</u> ≥ <u>of</u> ≠	

[11] [niks]	<u>is</u>	
[12] [naam]	<u>is</u>	[kleine letter] ⁿ⁺¹
[13] [NAAM]	<u>is</u>	[HOOFDLETTER] ⁿ⁺¹
[14] [label]	<u>is</u>	[cijfer] ⁿ⁺¹
[15] [nummer]	<u>is</u>	[label] <u>of</u> [label]: <u>of</u> [niks]
[16] []	<u>is</u>	[↻] <u>of</u> ; [symbool] ⁿ [↻]

[20] [ECOL-programma]	<u>is</u>	[FUN-beschrijving] ⁿ [nummer] START [] [programmadeel] ⁿ [nummer] KLAAR []
[21] [programmadeel]	<u>is</u>	[actie] <u>of</u> [SUB-beschrijving]
[22] [actie]	<u>is</u>	[opdracht] <u>of</u> [RIJ-reservering]
[23] [opdracht]	<u>is</u>	[nummer][toekenning] [] <u>of</u> [nummer][invoer/uitvoer] [] <u>of</u> [nummer][sprong] [] <u>of</u> [nummer][cyclus] [] <u>of</u> [nummer][SUB-roep] [] <u>of</u> [nummer][FUN-roep] [] <u>of</u> [nummer] []
[30] [toekenning]	<u>is</u>	[variabele] := [uitdr]
[31] [variabele]	<u>is</u>	[GP] <u>of</u> [GP-rij] ([uitdr])
[32] [GP]	<u>is</u>	[naam]
[33] [GP-rij]	<u>is</u>	[naam]
[34] [uitdr]	<u>is</u>	[voorteken][formule]
[35] [voorteken]	<u>is</u>	[niks] <u>of</u> + <u>of</u> -
[36] [formule]	<u>is</u>	[operand] <u>of</u> [formule][operator][operand]
[37] [operand]	<u>is</u>	[getal] <u>of</u> [variabele] <u>of</u> [FUN-roep] <u>of</u> ([uitdr])
[38] [getal]	<u>is</u>	[cijfer] ⁿ⁺¹ <u>of</u> [cijfer] ⁿ⁺¹ , <u>of</u> [cijfer] ⁿ⁺¹ . [cijfer] ⁿ⁺¹
[40] [invoer/uitvoer]	<u>is</u>	[variabele] := LEES <u>of</u> [variabele] := LEESYM <u>of</u> [GP-rij] := BESTAND([uitdr]) <u>of</u> [GP-rij] := BESTAND([uitdr],[uitdr],[uitdr]) <u>of</u> SPATIE <u>of</u> SPATIE([uitdr]) <u>of</u> NR <u>of</u> NR([uitdr]) <u>of</u> NP <u>of</u> TEKST := "[symbool] ⁿ⁺¹ " <u>of</u> SCHRIJF := [uitdr] <u>of</u> SCHRIJM := [uitdr] <u>of</u> SCHRIJFSYM := [uitdr] <u>of</u> SCHRIJF([uitdr],[uitdr]) := [uitdr] <u>of</u> BESTAND([uitdr]) := [GP-rij] <u>of</u> BESTAND([uitdr],[uitdr],[uitdr]) := [GP-rij]

[45] [sprong]	<u>is</u>	NAAR [label] <u>of</u> ALS[vergelijking] DAN [label] ANDERS [label] <u>of</u> ALS[vergelijking] DAN [label]
[50] [vergelijking]	<u>is</u>	[uitdr][vergelijker][uitdr] <u>of</u> [vergelijking] EN [vergelijking] <u>of</u> [vergelijking] OF [vergelijking] <u>of</u> ([vergelijking])
[55] [RIJ-reservering]	<u>is</u>	RIJ ([uitdr] : [uitdr])[GP-rij] [] <u>of</u> RIJSYM([uitdr] : [uitdr])[GP-rij] []
[60] [cyclus]	<u>is</u>	MET[uitdr], [GP] := [uitdr], [uitdr] [programmadeel] ⁿ [nummer]HERHAAL
[70] [SUB-roep]	<u>is</u>	[SUB-naam]
[71] [SUB-beschrijving]	<u>is</u>	[nummer]SUB[SUB-naam] [] [opdracht] ⁿ [nummer]END []
[72] [SUB-naam]	<u>is</u>	[naam]
[80] [FUN-roep]	<u>is</u>	[FUN-NAAM][lijst aktuelen] <u>of</u> [FUN-NAAM] <u>of</u> [FUN-naam][lijst aktuelen]
[81] [FUN-NAAM]	<u>is</u>	[NAAM]
[82] [FUN-naam]	<u>is</u>	[naam]
[83] [lijst-aktuelen]	<u>is</u>	([aktuelen])
[84] [aktuelen]	<u>is</u>	[aktuele] <u>of</u> [aktuelen] ,[aktuele]
[85] [aktuele]	<u>is</u>	[uitdr] <u>of</u> [GP-rij]
[86] [FUN-beschrijving]	<u>is</u>	[nummer]FUN[FUN-naam][lijst formelen] [] [opdracht] ⁿ [nummer]FUN := [uitdr] [] <u>of</u> [nummer] []
[87] [lijst formelen]	<u>is</u>	([formelen])
[88] [formelen]	<u>is</u>	[formeel] <u>of</u> [formelen] ,[formeel]
[89] [formeel]	<u>is</u>	[GP] <u>of</u> RIJ[GP-rij]